

面向 SQL 等价性判定的失效感知分层方法

摘要: 为应对有限预算下 SQL 等价性判定中形式化验证覆盖不足与模型判定边界不清的问题, 提出失效感知路由式 SQL 等价性判定方法 FAR-SQL (Failure-Aware Routing for SQL equivalence judgment)。传统形式化验证能够提供可信的确定结论, 但在复杂聚合、嵌套查询和方言函数等场景下容易产生超时、不支持或运行异常; 直接采用模型判定虽可提升覆盖能力, 却难以保留形式化验证的可信边界。FAR-SQL 以“形式化优先、失效后补全”为原则, 通过任务适配的本地监督微调模型、历史 VeriEQL 运行日志构建的验证收益画像, 以及面向非确定状态的失效感知路由, 形成形式化确定域与非确定补全域分离的分层判定机制。该方法根据历史运行日志估计不同 SQL 结构在多种时间预算下的验证收益, 为待判定查询对分配动态验证预算; 对 VeriEQL 返回的 equivalent 或 inequivalent 结论予以保留; 对 timeout、unsupported、runtime error 和 conversion error 等非确定状态进行归一化, 并调用经 SQL 等价任务适配的本地模型进行语义补全。实验在 Calcite+Spider-DAIL 域外测试集上进行, 其中 Calcite 包含 232 对等价样本, Spider-DAIL 包含 180 对不等价样本。实验表明, 固定预算 VeriEQL 在单调累计口径下的最高几何平均准确率为 38.46%, Qwen2.5-Coder-1.5B 原始模型为 43.77%; FAR-SQL 结合 Qwen2.5 监督微调模型达到 83.98%, 结合 Qwen3 监督微调模型达到 83.69%。结果说明, 动态预算分配与失效感知补全能够在保留形式化验证确定输出的同时, 提升有限预算下 SQL 等价性判定的覆盖能力和类别均衡性。

关键词: SQL 等价性判定; 形式化验证; 动态预算; 失效感知路由; 监督微调

文献标志码: A **中图分类号:** TP311

Failure-Aware Hierarchical Method for SQL Equivalence Judgment

Abstract: A failure-aware routing method for SQL equivalence judgment, FAR-SQL, is proposed to address insufficient formal-verification coverage and unclear model decision boundaries under bounded budgets. Formal verification can provide trustworthy determined conclusions, but it often produces timeout, unsupported, runtime-error, or conversion-error states for complex aggregation, nested queries, and dialect-specific functions. Direct model-based judgment can improve coverage, but it is difficult to preserve the trust boundary of formal verification. FAR-SQL follows a formal-first and completion-after-failure principle. It integrates a task-adapted local supervised fine-tuning model, verification-yield profiles learned from historical VeriEQL logs, and failure-aware routing over non-determined states, thereby separating the formal determined region from the non-determined completion region. Historical VeriEQL logs are used to characterize verification yields of different SQL structures under multiple time budgets and allocate dynamic verification budgets for incoming query pairs. Equivalent and inequivalent conclusions returned by VeriEQL are preserved, whereas non-determined states, including timeout, unsupported, runtime error, and conversion error, are normalized and routed to the SQL-equivalence-adapted local model for semantic completion. Experiments are conducted on the out-of-domain Calcite+Spider-DAIL test set, where Calcite contains 232 equivalent pairs and Spider-DAIL contains 180 inequivalent pairs. Fixed-budget VeriEQL reaches at most 38.46% geometric mean accuracy under the monotone cumulative protocol, and the base Qwen2.5-Coder-1.5B model reaches 43.77%. FAR-SQL achieves 83.98% with the Qwen2.5 supervised fine-tuning model and 83.69% with the Qwen3 supervised fine-tuning model. The results show that dynamic budget allocation and failure-aware completion improve coverage and class balance while preserving determined outputs of formal verification.

Key words: SQL equivalence judgment; formal verification; dynamic budgeting; failure-aware routing; supervised fine-tuning

SQL 是数据库系统中数据访问、业务分析和应用开发的核心接口。SQL 等价性判定旨在判断两个 SQL 查询在相同数据库模式与完整性约束下, 是否对任意合法数据库实例返回相同结果。该问题长期服务于查询优化、查询重写规则验证和数据库系统回归测试等基础任务^[1-2]。随着 Text-to-SQL 技术和大语言模型驱动的数据分析工具快速发展, SQL 等价性判定被进一步用于候选 SQL 比较、训练数据筛选、模型输

出评估以及系统升级后的行为一致性检查。因此, 在有限时延和可控资源预算下获得可信且覆盖充分的判定结果, 已成为数据库工具链中的重要需求。

现有 SQL 等价性判定方法主要面临可信性与覆盖性的矛盾。形式化验证是其中可信度较高的技术路线。Cosette、HoTTSQL、SPES、QED 和 VeriEQL 等工作分别从证明器编码、可检查语义、袋语义推理、反例生成和有界验证等方面推进了 SQL 等价验证研

究^[3-8]。其中, VeriEQL 能够在其支持范围内给出可解释、可复核的确定结论, 为 SQL 等价性判定提供了较强的可信基础。然而, 真实场景中的 SQL 查询往往包含复杂聚合、嵌套结构、方言函数和多样化数据库模式。在固定验证预算下, 形式化验证器容易出现 timeout、unsupported、runtime error 或 conversion error 等非确定状态, 使部分样本无法获得明确结论。实验中, 固定预算 VeriEQL 在测试集上的 GM 最高为 38.46%, 说明有限预算下形式化验证的主要瓶颈不在于确定结论的可信度, 而在于复杂 SQL 场景中的覆盖不足。

大语言模型为形式化验证未覆盖的样本提供了补充可能。已有研究表明, 大语言模型能够读取数据库模式和 SQL 文本, 并比较查询对之间的语义关系^[9-11]。但在批量评测、数据清洗和回归测试等高频场景中, 外部大模型或大参数模型通常会带来较高的推理成本、响应时延和数据出域风险, 难以稳定嵌入企业或科研机构内部的数据库工具链。轻量级本地模型具有部署灵活、推理开销低和数据不出域等优势, 更适合作为实际系统中的辅助判定组件。实验中, Qwen2.5-Coder-1.5B 原始模型的 GM 为 43.77%, 不等价样本准确率为 35.00%; 经过 SQL 等价判定任务监督微调后, Qwen2.5-Coder-1.5B 和 Qwen3-1.7B 的 GM 分别提升至 76.20% 和 77.88%。这表明, 任务适配能够改善轻量级本地模型在 SQL 等价判定中的类别均衡性和语义区分能力。

直接以模型判定替代形式化验证, 会削弱结论的可信边界; 而单一固定预算的形式化验证, 又难以覆盖复杂 SQL 查询对。SQL 等价性判定因此需要一种分层协同机制, 在有限预算内兼顾判定可信性、样本覆盖率与运行开销。为此, 提出面向 SQL 等价性判定的失效感知路由式分层方法 FAR-SQL (Failure-Aware Routing for SQL equivalence judgment)。FAR-SQL 并非简单串联 VeriEQL 与模型判定, 而是围绕“何时继续验证、何时停止等待、何种失效状态如何补全”建立统一路由: 离线阶段先利用形式化标注样本监督微调轻量级本地模型, 并从历史 VeriEQL 运行日志中学习不同 SQL 结构的验证收益画像; 在线阶段由失效感知路由器为查询对分配验证预算, 优先保留 VeriEQL 的 equivalent 或 inequivalent 确定结论, 仅在 timeout、unsupported、runtime error 或 conversion error 等非确定状态下调用状态感知语义补全器。通过将形式化验证与模型判定限定在各自更适合作用的范围内, FAR-SQL 在保

留可信边界的同时, 提高了有限预算下的判定覆盖能力, 并减少了不必要的模型推理开销。

为便于结果复现与后续比较, 相关代码、非确定状态归一化规则、预算配置和实验提示模板将随后公开。

本研究的主要贡献如下:

(1) 提出了面向 SQL 等价性判定的失效感知路由式分层框架 FAR-SQL。该框架将判定空间划分为形式化确定域和非确定补全域, 在保留 VeriEQL 确定输出的同时, 将模型判定限定在形式化验证无法覆盖的样本上。

(2) 构建了面向 SQL 等价性判定的轻量级本地监督微调模型。该模型利用形式化标注样本学习等价与不等价查询对的语义边界, 作为 VeriEQL 非确定区域的语义补全器, 缓解原始轻量级模型的类别偏置问题。

(3) 提出了基于历史 VeriEQL 运行日志的验证收益画像与动态预算路由机制。该机制刻画不同 SQL 结构在多种时间预算下的确定收益, 并据此决定在线验证预算, 使形式化验证资源优先分配给更可能产生确定结论的样本。

(4) 设计了面向非确定状态的状态归一化补全策略, 并在 Calcite+Spider-DAIL 域外测试集上进行验证。实验结果表明, FAR-SQL 优于固定预算 VeriEQL、原始模型、纯监督微调模型以及多种提示式消融方法, 验证了失效感知路由与语义补全的有效性。

1 相关工作

1.1 基于执行的 SQL 等价判定

基于执行的判定方法常用于 Text-to-SQL 评测、候选 SQL 比较和训练数据筛选等场景。Spider 提供了跨领域自然语言到 SQL 生成基准^[12], DIN-SQL 等方法进一步推动了复杂 SQL 生成中的分解式推理^[13]。在这类任务中, 字符串匹配难以识别不同写法下的语义等价 SQL, 因此执行结果逐渐成为更常用的评测依据。Distilled Test Suite 通过构造多组测试数据库提升执行评测的可靠性^[14]。然而, 执行判定本质上依赖有限数据库实例: 两个查询在测试实例上返回相同结果, 并不能保证其在所有合法数据库实例上均等价。因此, 基于执行的方法能够提供低成本经验信号, 但难以给出严格的 SQL 等价结论。

1.2 基于形式化方法的 SQL 等价验证

形式化方法试图从语义层面证明两个 SQL 查询

在所有合法数据库实例上的等价性,长期服务于查询优化、查询重写规则验证和异构系统迁移等场景^[1-2]。早期研究关注 SQL 形式语义、NULL 处理和不完整信息等基础问题^[15-16],后续工作逐渐将 SQL 等价验证转化为定理证明、逻辑可满足性判定或反例搜索问题。Cosette、HoTTSQL、SPES、基于线性整数算术的等价证明方法、QED 和 VeriEQL 等工作从可检查证明、袋语义推理、反例生成和有界验证等方面推进了 SQL 等价验证研究^[3-8]。这些方法能够在支持范围内提供可信的确定结论,但在复杂 SQL、方言适配和有限时间预算下仍可能产生 timeout、unsupported、runtime error 或 conversion error 等非确定输出。本文在保留 VeriEQL 确定结论的基础上,进一步关注非确定状态下的预算分配与补全问题。

1.3 基于大语言模型的 SQL 等价判定

大语言模型为 SQL 等价判定提供了新的补充路径。LLM-SQL-Solver、Rookies 等工作尝试利用大模型读取 schema、SQL 查询和提示指令,以判断查询对是否等价^[9-11]。这类方法对复杂 SQL 文本具有较好的适应性,能够处理部分形式化验证器暂未覆盖的输入形式。然而,在自动化评测、数据清洗和系统测试等批量场景中,外部大模型调用会带来推理成本、响应时延和数据出域等部署约束。相比之下,本地轻量级模型具有部署灵活、推理成本低和数据不出域等优势,更适合嵌入企业或学校内部的数据库工具链。但原始轻量级模型直接用于等价判定时,仍容易受到语义推理能力不足和类别偏置影响。基于此,本文采用经 SQL 等价任务适配的本地模型,并将其限定在 VeriEQL 非确定区域内使用,使其作为形式化验证未覆盖样本的语义补全组件。

2 问题定义与评价指标

给定数据库模式 S 、完整性约束 C ,以及两个 SQL 查询 q_1 和 q_2 ,SQL 等价性判定要求判断二者是否在所有满足 S 和 C 的合法数据库实例上返回相同结果。若对任意合法数据库实例 D , $q_1(D)$ 与 $q_2(D)$ 的结果多重集一致,则两个查询等价;若存在至少一个合法数据库实例使二者结果不同,则两个查询不等价。

本文将输出标签定义为 $y \in \{\text{equivalent}, \text{inequivalent}, \text{uncertain}\}$ 。其中, equivalent 和 inequivalent 分别表示等价与不等价,uncertain 表示在当前预算和策略下未输出确定 yes/no 结论。FAR-SQL 中可调用形式化验证器 V 和本地监督微

调模型 M : V 的确定输出作为形式化确定结论, M 仅作用于 V 未能确定的区域。

评价指标采用等价样本准确率 Acc_{eq} 、不等价样本准确率 Acc_{neq} 和几何平均准确率 GM 。 GM 定义为:

$$GM = (Acc_{eq} \cdot Acc_{neq})^{1/2} \quad (1)$$

GM 用于衡量判定器在等价类和不等价类上的整体均衡性。相比整体准确率, GM 对类别偏斜更敏感;当方法只偏向某一类别时,另一类别准确率的下降会使 GM 明显降低。因此,本文采用 GM 作为主要指标,以反映判定器对等价样本和不等价样本的综合识别能力。

3 FAR-SQL 失效感知路由式分层方法

有限预算下的批量 SQL 等价判定需要在判定时延和语义可靠性之间取得平衡。形式化验证器的确定结论具有较高可信性,但不同 SQL 结构获得确定结论所需的验证代价并不相同;统一长预算会使低收益样本占用过多求解时间,而过早转向模型判定又会削弱验证器在可确定区域中的作用。

因此,FAR-SQL 将判定过程组织为“收益画像—失效感知路由—状态感知补全”的分层流程。图 1 给出了 FAR-SQL 的整体框架:离线阶段构建两类基础组件,即 SQL 等价任务适配的轻量级本地模型和基于历史 VeriEQL 运行日志的验证收益画像;在线阶段,失效感知路由器首先依据画像为输入 SQL 对分配验证预算,并调用 VeriEQL 进行形式化验证。若 VeriEQL 返回 equivalent 或 inequivalent,则该结论直接进入形式化确定域;若返回 timeout、unsupported、runtime error 或 conversion error 等非确定状态,则经状态归一化后进入状态感知语义补全流程。由此,FAR-SQL 在保留形式化验证确定输出的同时,将本地模型限定在 VeriEQL 未覆盖区域内使用。为控制数据泄露风险,历史 VeriEQL 日志仅用于学习验证收益画像,监督微调数据也与 Calcite+Spider-DAIL 域外测试集保持来源隔离。

3.1 任务适配模型与收益画像

FAR-SQL 的在线判定依赖两个离线组件。第一个组件是 SQL 等价任务适配的轻量级本地模型,用于在形式化验证非确定时提供语义补全;第二个组件是验证收益画像,用于描述不同 SQL 结构在不同时间预算下获得确定结论的概率,为在线动态预算路由提供先验依据。

小模型的训练数据由两类形式化标注样本组成。第一类来自 VeriEQL 发布的 LeetCode 多解 SQL 数据, 包含 14 905 对等价样本和 3 586 对不等价样本。第二类基于 SynSQL 派生构造: SynSQL 提供 schema、自然语言问题和参考 SQL, 本文采用多个 Text-to-SQL 模型针对同一问题生成候选 SQL, 并在同一实例内部配对, 由 VeriEQL 进行有界等价判定, 最终得到 23 052 对等价样本和 399 811 对不等价样本。为提供推理监督, 进一步使用 Qwen3-32B 为带标签样本生成简短推理轨迹^[17-18], 并仅保留最终答案与 VeriEQL 标签一致的样本。随后按类别平衡采样, 形成 30 000 对等价样本和 30 000 对不等价样本, 作为监督微调数据集 D_{sft} 。训练目标为自回归交叉熵损失:

$$L_{SFT} = -E[\log p_{\theta}(r, y|x)] \quad (2)$$

其中, r 表示蒸馏得到的推理轨迹, y 表示等价性标签。该模型仅在 VeriEQL 返回非确定状态时调用。本文评估采用 Calcite+Spider-DAIL 域外测试集, 与训练数据保持来源隔离。

验证收益画像由历史 VeriEQL 运行日志构建。设 $\phi(q_1, q_2, S)$ 表示由查询对和 schema 抽取的结构特征, 历史日志中的每条记录表示为 (ϕ_i, si, τ_i) , 其中 si 为归一化验证状态, τ_i 为运行时间。对于画像组 G 和预算 t , 确定率定义为:

$$\text{cov}(G, t) = \frac{|D_{G,t}|}{|G|} \quad (3)$$

其中, $D_{G,t}$ 表示画像组 G 中在预算 t 内得到确定结论的历史样本集合。该画像刻画不同 SQL 结构的验证收益曲线, 并为后续失效感知路由选择验证预算提供依据。

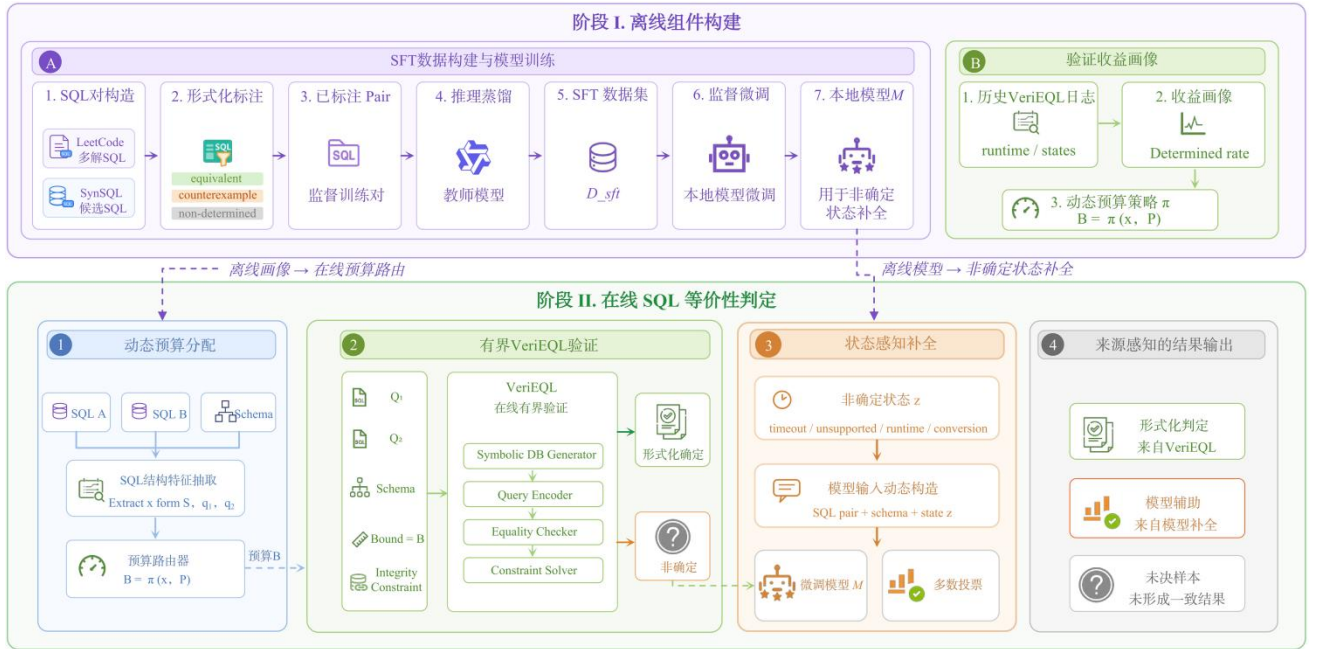


图 1 FAR-SQL 整体框架

Fig.1 Overall framework of FAR-SQL

3.2 失效感知路由与预算分配

在线判定时, 失效感知路由器根据输入 SQL 对的结构特征匹配验证收益画像, 并比较候选预算下的确定率和边际收益。若某类结构在短预算下已能保留主要验证收益, 则采用较短预算; 若历史上长期表现为低确定率或高 unsupported 风险, 则减少形式化等待, 使样本更早进入非确定补全流程。

预算确定后, FAR-SQL 在给定时间约束内调用

VeriEQL 进行有界验证。VeriEQL 的输出分为确定结论和非确定状态两类: 前者包括 equivalent 和 inequivalent, 表示验证器在当前预算内完成等价性判定; 后者包括 timeout、unsupported、runtime error、conversion error 和 unknown, 分别对应超时、语法或特性不支持、验证运行异常、输入转换失败以及其他未归类状态。对于确定结论, FAR-SQL 将其作为最终判定; 对于非确定状态, 系统记录状态类型和运行摘要, 并进入状态感知语义补全流程。

3.3 状态感知语义补全

对于 VeriEQL 未能给出确定判定结果的样本, FAR-SQL 依据归一化状态构造补全提示。不同状态对应不同的信息侧重: runtime error 保留必要的运行摘要, conversion error 强调原始 SQL 与 schema 信息, timeout 和 unsupported 采用与监督微调阶段一致的判定提示, 并加入精简的验证状态摘要。对于聚合类 timeout, 提示进一步关注分组粒度、聚合输入、过滤条件和去重语义等因素。

随后, FAR-SQL 调用监督微调后的本地模型进行状态感知语义补全。模型补全采用 Self-Consistency@3 策略^[19]: 每个样本独立生成 3 次候选结果, 并解析最终 yes/no 标签。若形成多数票, 则采用多数结果; 若无法形成多数票或不存在合法答案标签, 则返回 uncertain。

3.4 判定算法与复杂度分析

算法 1 失效感知分层 SQL 等价性判定

输入: 数据库模式 S , 查询对 $Q=(q_1, q_2)$, 验证收益画像 P , 验证器 V , 本地模型 M , 失效感知路由策略 π 。

输出: 判定标签 y 及其来源 c 。

1. 从 S 和 Q 中抽取结构特征 x ;
2. 由 P 和失效感知路由策略 π 为 x 分配验证预算 b ;
3. 若 $b>0$, 则运行 V , 得到状态 s ;
4. 若 s 为确定状态, 则返回 $(s, \text{形式化})$;
5. 对 s 归一化, 得到 z ;
6. 根据 z 和 x 选择状态感知补全提示 g ;
7. 构造包含 S 、 Q 、 z 和 g 的模型输入;
8. 调用 M 生成 K 个候选答案集合 A ;
9. 若 A 多数票稳定, 则返回(多数投票, 模型辅助);

算法 1 给出了 FAR-SQL 的在线判定流程, 其中 S 表示数据库模式, Q 表示待判定查询对, P 表示验证收益画像。该流程首先根据结构特征分配验证预算; 若 VeriEQL 在预算内返回 equivalent 或 inequivalent, 则直接保留形式化结论; 否则对非确定状态进行归一化, 并调用任务适配模型进行多次补全。只有当补全结果形成稳定多数票时, 系统才返回模型辅助标签, 否则输出 uncertain。

FAR-SQL 的额外开销主要来自 SQL 结构特征抽取和模型补全。结构特征抽取只依赖 SQL 文本和 schema; 模型补全仅在 VeriEQL 非确定样本上触发, 不增加形式化确定样本的推理成本。对于批量判定任务, VeriEQL 调用可以并行执行。

4 实验设置

4.1 数据集

主实验与 LLM-SQL-Solver^[9] 的评测设置保持一致, 采用 Calcite+Spider-DAIL 域外测试集。该测试集包含 412 对 SQL 样本, 其中 Calcite 部分包含 232 对等价样本, Spider-DAIL 部分包含 180 对不等价样本。监督微调数据来自 LeetCode 多解 SQL 和 SynSQL 派生样本, 与 Calcite+Spider-DAIL 测试集在来源上保持隔离, 用于考察 FAR-SQL 在非训练来源 SQL 对上的判定能力。

4.2 实验设置

实验比较三类方法。第一类为固定预算 VeriEQL, 仅调用形式化验证器, 预算设为 10 s、30 s、60 s 和 120 s; 若验证器未返回 equivalent 或 inequivalent 结论, 则记为未决。第二类为纯模型自一致性, 不调用验证器, 直接使用本地模型进行 SQL 等价判定, 并采用 Self-Consistency@3 的多数结果。第三类为 FAR-SQL, 先保留 VeriEQL 的确定输出, 再对非确定区域进行失效感知模型辅助补全。三类方法分别用于考察形式化验证器的覆盖能力、本地模型的独立判定能力, 以及二者在分层机制下的协同效果。

模型实验采用 Qwen2.5-Coder-1.5B 和 Qwen3-1.7B 两个本地轻量级模型系列^[20-21], 并比较原始模型与 SQL 等价任务监督微调 (supervised fine-tuning, SFT) 模型。组件消融实验包括两组: 一是比较 Minimal VeriEQL Signal、Full VeriEQL Context 与 FAR-SQL, 以检验直接增加验证器上下文是否能够替代非确定状态建模; 二是比较 Base backbone 与 SFT backbone 下的 FAR-SQL 结果, 以分析任务适配模型对非确定区域补全的作用。动态预算策略不并入组件消融, 而在质量与效率分析中单独比较统一固定预算与动态预算。

5 实验结果与分析

5.1 主结果

表 1 给出了 Calcite+Spider-DAIL 域外测试集上的总体结果。固定预算 VeriEQL 在该测试集上的主要限制来自覆盖不足。随着预算由 10 s 增加到 120 s, VeriEQL 的 GM 由 32.41% 提升至 38.46%, 但未决率仍为 53.40%。这说明增加验证时间能够带来一定收益, 但在复杂域外 SQL 上, 单纯延长固定预算难以充分扩大形式化验证覆盖范围。

纯模型 Self-Consistency@3 能够显著降低未决

率,但基础模型直接用于 SQL 等价判定时仍存在类别偏斜。例如, Qwen2.5-Base 的 Acc_{eq} 为 54.74%, Acc_{neq} 仅为 35.00%, GM 为 43.77%。经过 SQL 等价任务监督微调后,本地模型的判定均衡性明显改善, Qwen2.5-SFT 和 Qwen3-SFT 的 GM 分别达到 76.20% 和 77.88%。这一结果表明,任务适配能够提升轻量级模型对等价与不等价样本的区分能力,但纯模型方法仍缺少对形式化确定结论的利用。

FAR-SQL 在监督微调模型基础上进一步取得最优总体性能。结合 Qwen2.5-SFT 时, FAR-SQL 的 Acc_{eq} 、 Acc_{neq} 、 GM 和未决率分别为 85.78%、82.22%、

83.98% 和 0.00%; 结合 Qwen3-SFT 时, 四项指标分别为 84.05%、83.33%、83.69% 和 0.00%。相较于对应的纯监督微调模型, FAR-SQL 分别带来 7.78 和 5.81 个百分点的 GM 提升。更重要的是, FAR-SQL 的提升同时体现在等价类和不等价类上, 而不是依赖单一类别的偏向性改进。该结果说明, 形式化确定结论保留、非确定状态区分和任务适配模型补全能够形成互补: VeriEQL 在可确定区域提供可信输出, 本地模型集中处理 VeriEQL 未覆盖样本, 从而在有限预算下获得更高且更均衡的判定性能。

表 1 Calcite+Spider-DAIL 上的性能与未决率对比 (%)

Table 1 Performance and undetermined-rate comparison on Calcite+Spider-DAIL (%)

Method	Backbone	Und.	Acc_{eq}	Acc_{neq}	GM
Fixed-budget VeriEQL	VeriEQL@10s	59.71	21.98	47.78	32.41
Fixed-budget VeriEQL	VeriEQL@30s	55.58	28.02	48.33	36.80
Fixed-budget VeriEQL	VeriEQL@60s	53.64	30.17	48.33	38.19
Fixed-budget VeriEQL	VeriEQL@120s	53.40	30.60	48.33	38.46
Model-only Self-Consistency@3	Qwen2.5-Base	10.92	54.74	35.00	43.77
Model-only Self-Consistency@3	Qwen2.5-SFT	0.24	76.29	76.11	76.20
Model-only Self-Consistency@3	Qwen3-Base	0.73	67.67	73.89	70.71
Model-only Self-Consistency@3	Qwen3-SFT	0.00	81.47	74.44	77.88
FAR-SQL	Qwen2.5-SFT	0.00	85.78	82.22	83.98
FAR-SQL	Qwen3-SFT	0.00	84.05	83.33	83.69

5.2 消融实验

表 2 组件消融实验 (%)

Table 2 Component ablation studies (%)

Backbone	Variant	Acc_{eq}	Acc_{neq}	GM
(I) Impact of VeriEQL information usage				
Qwen2.5-SFT	Minimal VeriEQL Signal	75.86	82.22	78.98
	Full VeriEQL Context	75.43	87.22	81.11
	FAR-SQL	85.78	82.22	83.98
Qwen3-SFT	Minimal VeriEQL Signal	76.29	81.11	78.67
	Full VeriEQL Context	73.71	83.33	78.37
	FAR-SQL	84.05	83.33	83.69
(II) Effect of supervised fine-tuning				
Qwen2.5	Base + FAR-SQL	73.28	62.78	67.82
	SFT + FAR-SQL	85.78	82.22	83.98
Qwen3	Base + FAR-SQL	78.45	81.11	79.77
	SFT + FAR-SQL	84.05	83.33	83.69

表 2 从验证器信息使用方式和模型任务适配两个角度分析 FAR-SQL 的组件贡献。第 (I) 部分比较 Minimal VeriEQL Signal、Full VeriEQL Context 和 FAR-SQL。Minimal VeriEQL Signal 仅向模型提供简化验证器状态, Full VeriEQL Context 直接提供更完整的验证器上下文, 而 FAR-SQL 进一步引入状态归一化和失效感知路由。结果显示, 简单增加上下文并不稳定优于 FAR-SQL, 说明非确定状态需要被结构化

建模, 而不是作为普通文本上下文直接交给模型。

第 (II) 部分考察模型任务适配的作用。对于 Qwen2.5, Base+FAR-SQL 的 GM 为 67.82%, SFT+FAR-SQL 提升至 83.98%; 对于 Qwen3, GM 由 79.77% 提升至 83.69%。该结果表明, 非确定区域补全不仅依赖验证器状态信息, 也依赖本地模型对 SQL 等价判定任务的适配能力。FAR-SQL 的提升来自形式化确定结论保留、失效状态结构化处理和任务适配

模型补全的共同作用。

5.3 质量与效率分析

为进一步分析动态预算分配的质量—效率收益,在固定后续补全模型和三次多数投票规则的条件下,仅将 VeriEQL 阶段的预算策略替换为统一固定预算,并与动态预算策略进行比较。失效状态建模已在表 2 中消融,因此该实验只考察预算选择对判定质量与运行耗时的影响。图 2 给出了两个 SFT backbone 下统一预算与动态预算的质量—效率权衡。

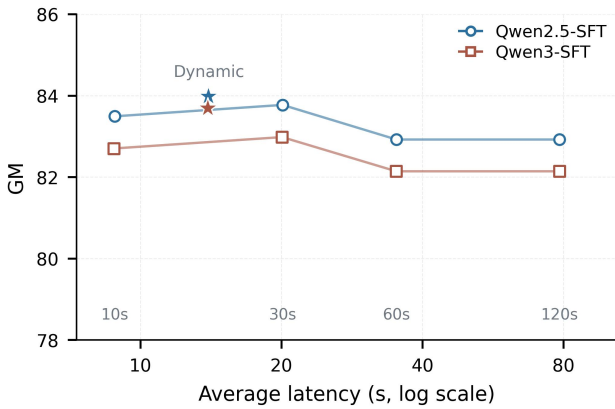


图 2 动态预算与统一预算的质量—效率权衡
Fig.2 Quality-efficiency trade-off between dynamic and uniform budgets

图 2 显示,动态预算策略在两个 SFT backbone 上均处于更优的质量—效率区域:相较统一固定预算,

动态预算在保持最高 GM 的同时显著降低平均运行开销。该结果说明, FAR-SQL 的优势并非来自简单延长 VeriEQL 等待时间,而是来自对验证预算的收益感知分配;它能够将形式化验证资源集中于更有收益的样本,并让难以验证的样本更快进入模型补全,从而在判定质量、运行效率和覆盖能力之间形成更稳定的折中。

表 3 比较了不同方法的判定质量与单样本耗时。纯模型 Self-Consistency@3 的平均耗时最低, Qwen2.5-SFT 和 Qwen3-SFT 分别为 0.51 s 和 0.47 s,但其 GM 仅为 76.20% 和 77.88%,低于 FAR-SQL。固定预算 VeriEQL 的耗时随预算增加显著上升,而 GM 在 60 s 附近趋于饱和: VeriEQL@60s 的 GM 为 38.19%,平均耗时为 35.00 s; VeriEQL@120s 的 GM 仅小幅提升至 38.46%,但平均耗时增加到 78.27 s。

FAR-SQL 在质量、效率和覆盖能力之间取得了更稳定的折中。结合 Qwen2.5-SFT 和 Qwen3-SFT 时, FAR-SQL 的 GM 分别为 83.98% 和 83.69%,平均耗时约为 14 s,明显低于 VeriEQL@60s 和 VeriEQL@120s,同时未决率降至 0。与纯模型相比, FAR-SQL 引入了一定形式化验证成本,但换取了更高的 GM 和可追踪的形式化确定来源。该结果表明,动态预算路由不是单纯的效率优化,而是保证形式化验证成本可控、同时维持补全质量的重要机制。

表 3 效率对比 (GM: %; 耗时: s)

Table 3 Efficiency comparison (GM: %, latency: s)

Method	Backbone / Budget	GM	Avg. (s)	P95 (s)
Model-only Self-Consistency@3	Qwen2.5-SFT	76.20	0.51	0.51
Model-only Self-Consistency@3	Qwen3-SFT	77.88	0.47	0.47
Fixed-budget VeriEQL	VeriEQL@10s	32.41	8.60	28.45
Fixed-budget VeriEQL	VeriEQL@30s	36.80	19.90	69.16
Fixed-budget VeriEQL	VeriEQL@60s	38.19	35.00	138.04
Fixed-budget VeriEQL	VeriEQL@120s	38.46	78.27	296.13
FAR-SQL	Qwen2.5-SFT	83.98	13.97	73.84
FAR-SQL	Qwen3-SFT	83.69	13.93	73.84

6 总结

针对有限预算下 SQL 等价性判定中形式化验证覆盖不足与模型判定可信边界不清的问题,提出了失效感知路由式分层方法 FAR-SQL。该方法以形式化验证器 VeriEQL 和轻量级本地模型为基础,构建了由任务适配模型、验证收益画像、失效感知路由和状

态感知语义补全组成的分层判定流程。对于 VeriEQL 能够给出确定结论的样本, FAR-SQL 保留其形式化输出;对于 timeout、unsupported、runtime error 和 conversion error 等非确定状态, FAR-SQL 先进行状态归一化,再调用监督微调后的本地模型完成语义补全。

为了评估 FAR-SQL 的有效性, 在已有研究使用的 Calcite+Spider-DAIL 域外测试集上进行了对比实验、组件消融和效率分析。实验结果表明, 固定预算 VeriEQL 在有限时延下覆盖不足, 原始本地模型直接判定时存在类别偏斜; 经过 SQL 等价任务监督微调后, 本地模型的判定均衡性得到明显改善。进一步结合 FAR-SQL 后, 方法在 GM 指标上优于固定预算 VeriEQL、纯模型 Self-Consistency@3 和多种组件消融设置, 并在约 14 s 平均耗时下实现 0 未决率。结果说明, 失效感知路由能够在保留形式化验证可信边界的同时, 提高有限预算下 SQL 等价性判定的覆盖能力与实用性。

参考文献:

- [1] BEGOLI E, CAMACHO-RODRIGUEZ J, HYDE J, et al. Apache Calcite: a foundational framework for optimized query processing over heterogeneous data sources[C]//Proceedings of the 2018 International Conference on Management of Data. New York: ACM, 2018: 221-230.
- [2] WANG Z, ZHOU Z, YANG Y, et al. WeTune: automatic discovery and verification of query rewrite rules[C]//Proceedings of the 2022 International Conference on Management of Data. New York: ACM, 2022: 94-107.
- [3] CHU S, WANG C, WEITZ K, et al. Cosette: an automated prover for SQL[C]//Proceedings of the 8th Biennial Conference on Innovative Data Systems Research. 2017: 1-7.
- [4] CHU S, WEITZ K, CHEUNG A, et al. HoTSQL: proving query rewrites with univalent SQL semantics[C]//Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation. New York: ACM, 2017: 510-524.
- [5] DING H, WANG Z, YANG Y, et al. Proving query equivalence using linear integer arithmetic[J]. Proceedings of the ACM on Management of Data, 2023, 1(4): 227:1-227:26.
- [6] HE Y, ZHAO P, WANG X, et al. VeriEQL: bounded equivalence verification for complex SQL queries with integrity constraints[J]. Proceedings of the ACM on Programming Languages, 2024, 8(OOPSLA1): 1071-1099.
- [7] WANG S, PAN S, CHEUNG A. QED: a powerful query equivalence decider for SQL[J]. Proceedings of the VLDB Endowment, 2024, 17(11): 3602-3614.
- [8] ZHOU Q, ARULRAJ J, NAVATHE S B, et al. SPES: a symbolic approach to proving query equivalence under bag semantics[C]//2022 IEEE 38th International Conference on Data Engineering. Piscataway: IEEE, 2022: 2735-2748.
- [9] ZHAO F, CHEN J, LIM L, et al. LLM-SQL-Solver: can LLMs determine SQL equivalence?[EB/OL]. [2026-05-29]. <https://arxiv.org/abs/2312.10321>.
- [10] SINGH R, BEDATHUR S. Can the rookies cut the tough cookie? Exploring the use of LLMs for SQL equivalence checking[EB/OL]. [2026-05-29]. <https://arxiv.org/abs/2412.05561>.
- [11] ZENG Q, MA S, NIKNAFS A, et al. Taming SQL complexity: LLM-based equivalence evaluation for text-to-SQL[EB/OL]. [2026-05-29]. <https://arxiv.org/abs/2506.09359>.
- [12] YU T, ZHANG R, YANG K, et al. Spider: a large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task[C]//Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. Brussels: Association for Computational Linguistics, 2018: 3911-3921.
- [13] POURREZA M, RAFIEI D. DIN-SQL: decomposed in-context learning of text-to-SQL with self-correction[C]//Advances in Neural Information Processing Systems. Red Hook: Curran Associates, 2023, 36: 36339-36348.
- [14] ZHONG R, YU T, KLEIN D. Semantic evaluation for text-to-SQL with distilled test suites[C]//Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. Stroudsburg: Association for Computational Linguistics, 2020: 396-411.
- [15] NEGRI M, PELAGATTI G, SBATELLA L. Formal semantics of SQL queries[J]. ACM Transactions on Database Systems, 1991, 16(3): 513-534.
- [16] IMIELINSKI T, LIPSKI W. Incomplete information in relational databases[J]. Journal of the ACM, 1984, 31(4): 761-791.
- [17] WEI J, WANG X, SCHUURMANS D, et al. Chain-of-thought prompting elicits reasoning in large language models[C]//Advances in Neural Information Processing Systems. Red Hook: Curran Associates, 2022, 35: 24824-24837.
- [18] HSIEH C Y, LI C L, YE H C K, et al. Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes[C]//Findings of the Association for Computational Linguistics: ACL 2023. Toronto: Association for Computational Linguistics, 2023: 8003-8017.
- [19] WANG X, WEI J, SCHUURMANS D, et al. Self-consistency improves chain of thought reasoning in language models[EB/OL]. [2026-05-29]. <https://arxiv.org/abs/2203.11171>.
- [20] HUI B, YANG J, CUI Z, et al. Qwen2.5-Coder technical report[EB/OL]. [2026-05-29]. <https://arxiv.org/abs/2409.12186>.
- [21] YANG A, LI A, YANG B, et al. Qwen3 technical report[EB/OL]. [2026-05-29]. <https://arxiv.org/abs/2505.09388>.